Menus

Menus are a common user interface component in many types of applications. To provide a familiar and consistent user experience, you should use the Menu APIs to present user actions and other options in your activities.

There are three fundamental types of menus or action presentations on all versions of Android as:-

1. Options menu and app bar
2. Context menu and contextual action mode
3. Popup menu

1. Options menu and app bar: The options menu is the primary collection of menu items for an activity. It's where you should place actions that have a global impact on the app, such as "Search," "Compose email," and "Settings."

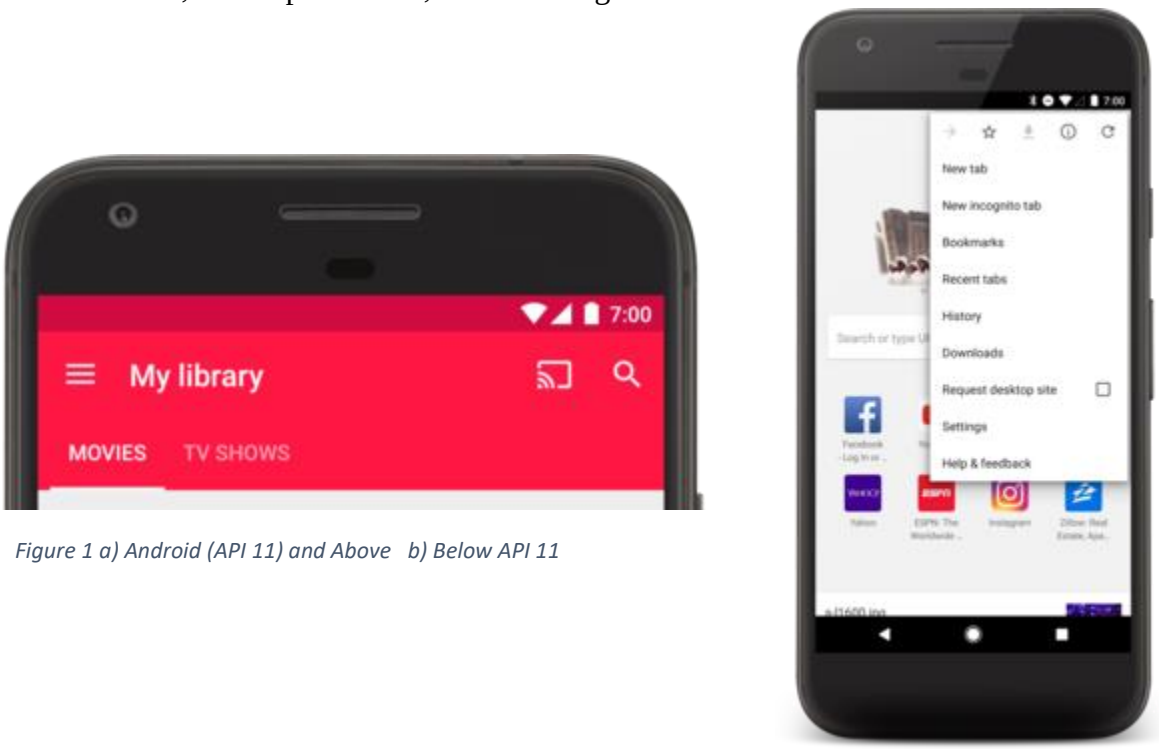
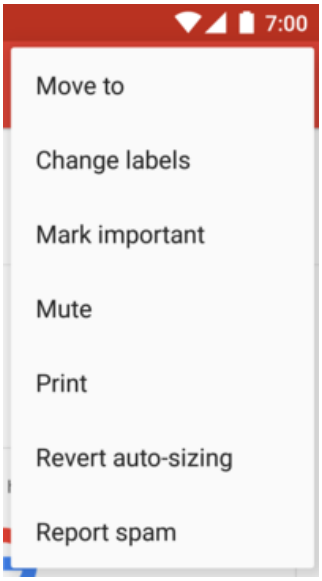


Figure 1 a) Android (API 11) and Above b) Below API 11

2. Context menu and contextual action mode: A context menu is a floating menu that appears when the user performs a long-click on an element. It provides actions that affect the selected content or context frame.
   The contextual action mode displays action items that affect the selected content in a bar at the top of the screen and allows the user to select multiple items.

3. Popup menu: A popup menu displays a list of items in a vertical list that's anchored to the view that invoked the menu. It's good for providing an overflow of actions that relate to specific content or to provide options for a second part of a command. Actions in a popup menu should not directly affect the corresponding content that's what contextual actions are for. Rather, the popup menu is for extended actions that relate to regions of content in your activity.



How to create a Menu?

For all menu types mentioned above, Android provides a standard XML format to define menu items. Instead of building a menu in your activity's code, you should define a menu and all its items in an XML menu resource. You can then inflate the menu resource i.e. load the XML files as a Menu object in your activity.

When creating menu file you need to know the following tags

<menu>

It defines a Menu, which is a container for menu items. A <menu> element must be the root node for the file and can hold one or more <item> and <group> elements.

<item>

It creates a MenuItem, which represents a single item in a menu. This element may contain a nested <menu> element in order to create a submenu.

<group>

It is an optional, invisible container for <item> elements. It allows you to categorize menu items so they share properties such as active state and visibility.

Example,

```
<?xml version="1.0" encoding="utf-8"?>
  <menu xmlns:android="http://schemas.android.com/apk/res/android">
     <item android:id="@+id/i1"
          android:title="item"
          android:icon="@drawable/item" >

        <!-- "item" submenu -->
        <menu>
           <item android:id="@+id/a"
                android:title="subitem a"
                android:icon="@drawable/subitem_a"/>
           <item android:id="@+id/b"
                android:title="subitem b"
                android:icon="@drawable/subitem_b" />
        </menu>
     </item>
  </menu>
```

The <item> element supports several attributes you can use to define an item's appearance and behavior. The items in the above menu include the following attributes:

android:id
A resource ID that's unique to the item, which allows the application to recognize the item when the user selects it.

android:icon
A reference to a drawable to use as the item's icon.

android:title
A reference to a string to use as the item's title.

Making an Option Menu
To make an option menu, we need to Override onCreateOptionsMenu() method as follows:

```
@Override
  public boolean onCreateOptionsMenu(Menu menu) {
  MenuInflater inflater = getMenuInflater();
     inflater.inflate(R.menu.menu_file, menu);
     return true;
  }
```

MenuInflater inflater = getMenuInflater();
This gives a MenuInflater object that will be used to inflate (convert our XML file into Java Object)

Handling Click Events

When the user selects an item from the options menu, the system calls your activity's onOptionsItemSelected() method. This method passes the MenuItem selected. You can identify the item by calling getItemId() method, which returns the unique ID for the menu item (defined by the android:id attribute in the menu resource). You can match this ID against known menu items to perform the appropriate action.

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {
   //Handle item selection
   switch (item.getItemId()) {
      case R.id.i1:
         //perform any action;
         return true;
      case R.id.a:
          //perform any action;
         return true;
      case R.id.b:
          //perform any action;
```

```
            return true;
        default:
            return super.onOptionsItemSelected(item);
    }
}
```

Making Contextual Menu

To make a floating context menu, you need to follow the following steps:

Register the View to which the context menu should be associated by calling registerForContextMenu() and pass it the View.

If your activity uses a ListView or GridView and you want each item to provide the same context menu, you can register your all items for a context menu by passing the ListView or GridView object to registerForContextMenu() method.

Implement the onCreateContextMenu() method in your Activity.

When the registered view receives a long-click event, the system calls your onCreateContextMenu() method. This is where you define the menu items, usually by inflating a menu resource. For example:

```
@Override
public void onCreateContextMenu(ContextMenu menu, View v,
                                ContextMenuInfo menuInfo) {
    super.onCreateContextMenu(menu, v, menuInfo);
    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.menu_file, menu);
}
```

Handling Click Events

When the user selects a menu item, the system calls onContextItemSelected() method so that you can perform the appropriate action. For example:

```
@Override
  public boolean onContextItemSelected(MenuItem item) {
      switch (item.getItemId()) {
          case R.id.i1:
              //Perform any action;
              return true;
          case R.id.a:
              //Perform any action;
              return true;
          case R.id.b:
```

```
            //Perform any action;
            return true;
        default:
            return super.onContextItemSelected(item);
    }
}
```

Making Popup Menu

If you have define your menu.xml file in XML, here's how you can show the popup menu:

Make an object of PopupMenu, whose constructor takes the current application Context and the View to which the menu should be anchored.
Use MenuInflater to inflate your menu resource into the Menu object returned by PopupMenu.getMenu()
Call PopupMenu.show()
For example, here's a button that shows a popup menu when clicked on it:

```
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Button"
    android:onClick="pop" />
```

The activity can then show the popup menu like this:

```
public void pop(View v){
        PopupMenu popup = new PopupMenu(this,v);
        MenuInflater inflater = getMenuInflater();
        inflater.inflate(R.menu.menu_file,popup.getMenu());
        popup.show();
    }
```

Handling Click Events

To perform an action when the user selects a menu item, you must implement the PopupMenu.OnMenuItemClickListener interface to your Activity and register it with your PopupMenu by calling setOnMenuItemclickListener(). When the user selects an item, the system calls the onMenuItemClick() method in your Activity.

```
public void pop(View v){
        PopupMenu popup = new PopupMenu(this,v);
```

```
        popup.setOnMenuItemClickListener(this);
        MenuInflater inflater = getMenuInflater();
        inflater.inflate(R.menu.menu_file,popup.getMenu());
        popup.show();
    }

@Override
public boolean onMenuItemClick(MenuItem item) {
    switch (item.getItemId()) {
        case R.id.i1:
            //Perform any action;
            return true;
        case R.id.a:
            //Perform any action;
            return true;
        case R.id.b:
            //Perform any action;
            return true;
        default:
            return false;
    }
}
```